



OPERACIÓN KANTABILE

PROYECTO INTERMODULAR

ADRIÁN VICENTE LÓPEZ

EDUARDO PIQUER SANCHEZ

-CONTENIDO-

1. Introducción	4
2. Descripción del proyecto	4
2.1 Usuarios del sistema	5
2.2 Funcionalidades para usuarios	5
2.3 Funcionalidades para administradores	6
3. Objetivos del proyecto	6
4. Tecnologías utilizadas	6
5. Organización del equipo	7
6. Planificación inicial	8
7. Arquitectura del sistema	8
8. Estado final y viabilidad	9
9. Modelo de base de datos	9
8.1 Integridad y Persistencia	10
8.2 Relaciones y esquema E/R	10
10. Lógica base	10
10.1. Algoritmo de Organización y Normalización	11
10.2. Estructura de Directorios Dinámica	11
10.3. Gestión de la Integridad	11
10.4. Explicación Técnica de Funcionamiento	12
10.5. Ejemplo del Flujo de Nombres	12
11. Motor de Ejecución y Gestión de Sesión	13
11.1. Gestión Dinámica de Colas Privadas	13
11.2. Arquitectura del Reproductor Dual	14
11.3. Motor de Sincronización Avanzada	14
11.4. Interfaz de Escenario	15
12. Navegación y Flujo de Trabajo	15
12.1. Mapa del Sitio	15
12.2. Descripción de los Flujos Principales	15
12.3. Diseño de la Interfaz	16
13. Conectividad e Infraestructura de Comunicación	19
13.1. Comunicación Cliente-Servidor	19
13.2. Gestión de Recursos Multimedia	19
13.3. Conectividad entre Contextos	19

13.4. Automatización de Infraestructura y Despliegue Continuo.....	19
13.4.1 Crear archivo de instrucciones.....	20
13.4.2 Guardar las contraseñas en secreto.....	21
13.4.3 Funcionamiento y programación del archivo main.yml.....	22
13.5. Tareas Programadas y Automatización de Permisos en el Servidor.....	23
13.5.1. El problema de los permisos en un proyecto vivo	23
13.5.2. La solución: Un revisor automático paso a paso.....	24
14. Manual de incorporación de canciones al sistema de karaoke.....	25
14.1. Obtención de la canción	25
14.2. Procesamiento de audio con Moises	26
14.3. Generación y sincronización de la letra	28
14.4. Alta de la canción en el sistema de administración.....	32
14.5. Comprobación final.....	34
15. Alternativa de pago para generación automática de karaoke: Youka	34
16. Problemas y retos surgidos durante el desarrollo.....	36
17. Usuarios de prueba.....	38
18. Reflexión final	38
19. Bibliografía	39
20. Agradecimientos	40

1. Introducción

Este documento presenta el desarrollo y resultado final de **Kantabile**, una aplicación web de karaoke diseñada para ofrecer una experiencia similar a la de un karaoke tradicional en un entorno digital. La plataforma permite a los usuarios reproducir canciones con vídeo y letra sincronizada, facilitando una experiencia interactiva y accesible desde cualquier dispositivo con acceso a internet.

La aplicación ofrece un catálogo de canciones que los usuarios pueden consultar y añadir a una cola de reproducción personal. Cada usuario dispone de su propio reproductor y puede gestionar su cola de canciones, añadiendo nuevos temas o modificando su orden de reproducción según sus preferencias. De esta forma, **Kantabile** funciona como un karaoke personal que puede utilizarse tanto de forma individual como en grupo.

Además, la plataforma incorpora un sistema de peticiones mediante el cual los usuarios pueden solicitar la incorporación de nuevas canciones al catálogo. Estas solicitudes son gestionadas por los administradores, quienes pueden revisarlas y decidir su inclusión en el sistema.

Cada usuario dispone también de una página de perfil desde la que puede gestionar la información de su cuenta, incluyendo la modificación de su contraseña y otros datos personales.

Por otro lado, **Kantabile** cuenta con un panel de administración que permite gestionar las canciones disponibles, administrar las solicitudes recibidas y controlar los usuarios registrados en la plataforma. Los administradores también pueden acceder al reproductor y utilizar la aplicación como usuarios convencionales.

En esta memoria se describen los objetivos del proyecto, las tecnologías empleadas durante el desarrollo, la arquitectura de la aplicación, el proceso de implementación y los resultados obtenidos.

Puede accederse a la web a través de todos los logos del proyecto en este documento o del siguiente [enlace](#).

2. Descripción del proyecto

La aplicación consiste en una plataforma web de karaoke que permite a los usuarios reproducir canciones con vídeo y letra sincronizada. El sistema está diseñado para que cada usuario pueda disfrutar de una experiencia de karaoke personal, gestionando su propia cola de reproducción y seleccionando las canciones que desea interpretar.

La plataforma se divide en dos tipos principales de usuarios: usuarios normales y administradores, cada uno con diferentes funcionalidades dentro del sistema.

2.1 Usuarios del sistema

El sistema cuenta con dos tipos de usuarios:

Usuarios normales

Son los usuarios que utilizan la plataforma para disfrutar del karaoke. Estos usuarios pueden acceder al reproductor, gestionar su cola de canciones y realizar peticiones de nuevas canciones que aún no se encuentren en el sistema.

Administradores

Los administradores son los encargados de gestionar el contenido y el funcionamiento del sistema. Tienen acceso a herramientas de administración que les permiten añadir nuevas canciones, gestionar usuarios y revisar las peticiones realizadas por los usuarios. Además, los administradores también pueden utilizar el reproductor y el resto de funcionalidades como si fueran usuarios normales.

2.2 Funcionalidades para usuarios

Los usuarios registrados en la plataforma disponen de diferentes funcionalidades que les permiten interactuar con el sistema.

Reproductor de karaoke

Los usuarios pueden acceder a una página de reproductor donde se muestran las canciones disponibles en el sistema. Desde esta página pueden añadir canciones a su cola de reproducción personal. El reproductor se encarga de reproducir las canciones seleccionadas mostrando el vídeo correspondiente junto con la letra sincronizada.

Gestión de la cola de canciones

Cada usuario dispone de su propia cola de reproducción. Dentro de esta cola puede añadir canciones, eliminar canciones o modificar el orden en el que se reproducirán.

Página de peticiones

Si un usuario desea cantar una canción que no está disponible en el sistema, puede realizar una solicitud desde la página de peticiones para que dicha canción sea añadida al catálogo.

Perfil de usuario

Cada usuario cuenta con una página de perfil desde la que puede consultar y modificar su información personal, como por ejemplo su contraseña u otros datos de su cuenta.

2.3 Funcionalidades para administradores

Los administradores disponen de un panel de gestión que les permite administrar el contenido del sistema.

Gestión de canciones

Los administradores pueden añadir nuevas canciones al sistema, así como editar o eliminar canciones existentes.

Gestión de usuarios

Desde el panel de administración se pueden gestionar los usuarios registrados en la plataforma, permitiendo consultar información, modificar datos o realizar otras acciones de administración.

Gestión de peticiones

Los administradores pueden revisar las peticiones de canciones realizadas por los usuarios. A partir de estas solicitudes pueden decidir si añadir nuevas canciones al sistema.

3. Objetivos del proyecto

Objetivo general

Desarrollar una plataforma web que permita ofrecer una experiencia de karaoke accesible desde el navegador, facilitando la reproducción de canciones y la gestión del contenido del sistema.

Objetivos específicos

- Diseñar una interfaz sencilla e intuitiva para los usuarios.
- Implementar un sistema que permita organizar y reproducir canciones.
- Facilitar la incorporación de nuevas canciones mediante un sistema de peticiones.
- Desarrollar herramientas de administración para la gestión del contenido.
- Garantizar una experiencia de uso fluida y accesible para los usuarios.

4. Tecnologías utilizadas

Para el desarrollo de la aplicación web se han seleccionado tecnologías y herramientas que permiten un desarrollo ágil, flexible y compatible con la mayoría de dispositivos. A continuación, se detallan las principales tecnologías empleadas.

Entorno de desarrollo

VSCode: editor de código utilizado para programar tanto el frontend como el backend.

XAMPP: servidor local con Apache y **MySQL** que permite ejecutar y probar la aplicación durante el desarrollo.

Frontend

HTML y CSS: para estructurar y diseñar la interfaz web.

Bootstrap: framework de CSS que facilita la creación de interfaces responsivas y consistentes.

JavaScript: se utilizará de forma complementaria para añadir interactividad y mejorar la experiencia del usuario cuando sea necesario.

Backend

PHP: lenguaje de servidor utilizado para procesar las peticiones del usuario, gestionar la base de datos y controlar la lógica de la aplicación.

Base de datos

MySQL: sistema de gestión de base de datos relacional que almacena información de usuarios, canciones, peticiones y otros datos relevantes del sistema.

Control de versiones y colaboración

Git y GitHub: el control de versiones se gestiona mediante **Git**, utilizando VSCode para interactuar con **GitHub**. Esto permite centralizar el código, gestionar revisiones y facilitar la colaboración entre los miembros del equipo de manera organizada.

Herramientas de apoyo

Durante el desarrollo se han utilizado herramientas de inteligencia artificial como apoyo en ciertas tareas de programación y documentación.

La combinación de estas tecnologías y herramientas permite crear un sistema web funcional, escalable y compatible con distintos dispositivos, asegurando un desarrollo colaborativo, organizado y ágil.

Cabe destacar que la responsabilidad final sobre la arquitectura del sistema, la toma de decisiones críticas, la maquetación visual y la integración de la base de datos ha recaído íntegramente en los desarrolladores. De este modo, la Inteligencia Artificial se ha consolidado en este proyecto como un catalizador de productividad, permitiendo alcanzar un despliegue seguro, robusto y profesional en un menor intervalo de tiempo.

5. Organización del equipo

El desarrollo de **Kantabile** se realiza en un equipo de dos integrantes. Debido al tamaño reducido del equipo, ambos participantes colaboran de manera flexible en todas las partes del proyecto, incluyendo el diseño de la base de datos, la interfaz web y la lógica de la aplicación.

Esta forma de trabajo permite distribuir las tareas según las necesidades del proyecto y avanzar de manera ágil, realizando revisiones constantes del trabajo de manera colaborativa.

6. Planificación inicial

El desarrollo de **Kantabile** se organiza de forma **flexible y colaborativa**, adaptándose a la disponibilidad del equipo y a las necesidades que vayan surgiendo durante el proyecto. Aunque no se dispone de un cronograma rígido, sí se ha definido un conjunto de tareas principales que estructuran el trabajo y facilitan el avance progresivo del sistema:

Tareas principales

- Diseño y creación de la base de datos.
- Desarrollo de la interfaz de usuario, incluyendo las páginas del reproductor, perfil y peticiones e interfaz de administración.
- Implementación de la lógica de usuario y de administrador.
- Gestión y administración de canciones y peticiones.
- Integración del reproductor con vídeo y letra sincronizada.
- Pruebas, ajustes y mejoras de la experiencia de usuario.

Gestión del trabajo y seguimiento

El equipo mantiene un **registro de tareas pendientes y completadas**, permitiendo priorizar actividades y coordinar el trabajo de manera eficiente. Este enfoque flexible asegura que el proyecto avance de forma constante, incluso adaptándose a cambios o necesidades que puedan surgir durante el desarrollo.

7. Arquitectura del sistema

La arquitectura de **Kantabile** se basa en un modelo **cliente-servidor**. Los usuarios interactúan con la plataforma a través del navegador web, utilizando su reproductor personal y gestionando su cola de canciones.

El servidor, desarrollado en PHP, procesa las solicitudes de los usuarios y se comunica con la base de datos MySQL, que almacena información sobre usuarios, canciones y peticiones. El sistema incluye además una interfaz de administración que permite a los administradores gestionar canciones, revisar solicitudes y administrar usuarios.

Esta arquitectura modular y flexible facilita la integración de nuevas funcionalidades y asegura un desarrollo organizado y mantenible del proyecto.

8. Estado final y viabilidad

El proyecto Kantabile ha sido desarrollado y completado satisfactoriamente, implementando todas las funcionalidades principales previstas durante la fase de planificación. La aplicación dispone de una base de datos completamente funcional y de las distintas interfaces destinadas tanto a usuarios como a administradores.

Entre las funcionalidades implementadas se encuentran la reproducción de canciones con vídeo y letra sincronizada, la gestión de colas de reproducción personalizadas, el sistema de peticiones de nuevas canciones, la gestión de perfiles de usuario y el panel de administración para la gestión de contenidos y usuarios.

La utilización de tecnologías consolidadas como PHP, MySQL y Bootstrap ha permitido desarrollar una aplicación estable, mantenible y escalable. Asimismo, el uso de herramientas de desarrollo y control de versiones como Visual Studio Code y GitHub ha facilitado la organización del trabajo y el seguimiento de los cambios realizados durante el desarrollo.

Los resultados obtenidos demuestran la viabilidad técnica del proyecto y confirman que la aplicación cumple los objetivos establecidos inicialmente, ofreciendo una plataforma funcional y preparada para futuras ampliaciones o mejoras.

9. Modelo de base de datos

El sistema utiliza un sistema de gestión de bases de datos relacionales (RDBMS) **MariaDB** gestionado mediante la interfaz **phpMyAdmin**. Esta elección permite mantener la integridad referencial entre los usuarios, el catálogo musical y el estado del sistema en tiempo real (cola de reproducción).

La base de datos, denominada **karaoke**, se compone de cuatro tablas interconectadas mediante claves foráneas (Foreign Keys):

usuarios: Gestiona el acceso al sistema. Almacena el nombre, email, contraseña y el rol (Usuario o Administrador). Incluye un campo estado para el control de bloqueos de cuenta.

canciones: Es el núcleo del catálogo. No solo guarda metadatos (título, artista, estilo), sino que gestiona las rutas de sistema de archivos para los tres componentes esenciales del karaoke: el audio con **voz**, la pista **instrumental** y el archivo de **letra** (formato **.lrc**).

cola: Actúa como la tabla dinámica de la sesión. Relaciona qué usuario ha pedido qué canción, permitiendo gestionar el orden de actuación.

peticiones: Permite a los usuarios solicitar canciones que aún no están en el catálogo, incluyendo un control de estado (pendiente o realizado) y marca de tiempo.

8.1 Integridad y Persistencia

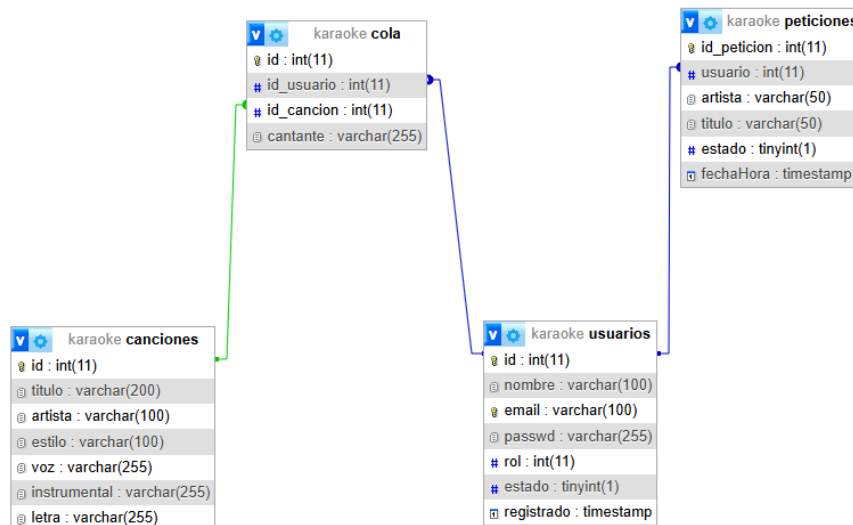
Se han implementado restricciones de integridad para garantizar que, si un usuario o canción se elimina, las entradas correspondientes en la cola o en peticiones se gestionen automáticamente (mediante ON DELETE CASCADE), evitando registros huérfanos.

8.2 Relaciones y esquema E/R

Usuarios → **Cola**: Una línea que salga de usuarios.id hacia cola.id_usuario. (Relación 1:N).

Canciones → **Cola**: Una línea que salga de canciones.id hacia cola.id_cancion. (Relación 1:N).

Usuarios → **Peticiones**: Una línea que salga de usuarios.id hacia peticiones.usuario. (Relación 1:N).



10. Lógica base

La lógica central del proyecto ha sido diseñada para actuar como un middleware inteligente entre la base de datos MariaDB y el sistema de archivos del servidor. El objetivo es garantizar que la biblioteca musical sea **autónoma, escalable y libre de errores de redundancia**.

10.1. Algoritmo de Organización y Normalización

El sistema no almacena los archivos de forma plana, sino que sigue una estructura jerárquica basada en metadatos para optimizar la búsqueda y el mantenimiento.

Normalización Estricta de Cadenas: Se utiliza la función `preg_replace('/[^A-Za-z0-9\ - ]/', '', $string)` para sanear los nombres. Esta técnica es fundamental para eliminar caracteres que el sistema de archivos podría interpretar como comandos o rutas (como `../` o símbolos de sistema), garantizando la portabilidad entre servidores Linux y Windows.

Tratamiento de Espacios: Se aplica `str_replace(' ', '_', $string)`. El uso de guiones bajos en lugar de espacios evita los errores de "espacios escapados" (`%20`) en las URLs, lo que facilita la carga directa de los recursos en el reproductor HTML5.

Prevención de Colisiones mediante Entropía Temporal: Se añade un timestamp dinámico a cada archivo. Esto asegura que, si dos versiones de la misma canción son subidas, el sistema genere nombres únicos, evitando la pérdida de datos por sobreescritura accidental.

10.2. Estructura de Directorios Dinámica

El sistema implementa una arquitectura de almacenamiento por nodos de artista, gestionada mediante las siguientes funciones nativas de PHP:

Verificación de Existencia (`is_dir`): Antes de cada operación de escritura, el script comprueba si la rama del árbol de directorios ya existe, evitando errores de duplicidad.

Creación Recursiva (`mkdir`): Si el artista es nuevo en el catálogo, se ejecuta `mkdir($ruta, 0777, true)`. El parámetro `true` es crítico, ya que permite la creación de carpetas anidadas de forma recursiva en un solo paso, mientras que el permiso `0777` asegura que el servidor tenga privilegios de lectura/escritura sobre los nuevos **assets**.

Distribución de Componentes: Se segmentan los recursos en `/uploads/canciones/` para flujos binarios de audio y `/uploads/letras/` para archivos de texto sincronizado, manteniendo una separación de intereses clara en el servidor.

10.3. Gestión de la Integridad

Se ha implementado una lógica de "borrado inteligente" para mantener una sincronización perfecta entre la base de datos y el almacenamiento físico (Estado Espejo).

Sincronización de Borrado (`unlink`): Al eliminar un registro, el sistema no solo borra la fila en SQL, sino que utiliza las rutas almacenadas para invocar la función `unlink()`. Esto libera espacio en el disco de forma inmediata.

Análisis de Residuos (scandir): Tras la eliminación de los archivos, se escanea el directorio del artista. El sistema cuenta los elementos devueltos por scandir(), ignorando los punteros de navegación de sistema (. y ..).

Recursividad Inversa de Limpieza (rmdir): Si el contador de archivos es cero, el sistema interpreta que el artista ya no tiene canciones en el catálogo y procede a eliminar la carpeta física con rmdir(). Esto evita la acumulación de carpetas vacías (directorios huérfanos) que degradarían el rendimiento del sistema de archivos a largo plazo.

10.4. Explicación Técnica de Funcionamiento

Para entender el proceso completo desde la interfaz hasta el disco, el flujo se divide en cuatro etapas técnicas:

Etapas de Captura: El formulario **multipart/form-data** envía los arrays globales \$_POST (datos) y \$_FILES (binarios). El sistema valida que los archivos no superen el límite de subida del servidor (**upload_max_filesize**).

Etapas de Saneamiento: Se limpian los nombres del artista y título. Se escapan las cadenas con **mysqli_real_escape_string()** para asegurar que caracteres como el apóstrofe (ej. "Guns N' Roses") no corrompan la sentencia SQL posterior.

Etapas de Persistencia Física: Se utiliza **move_uploaded_file()**. Esta función es una medida de seguridad vital en PHP, ya que verifica que el archivo que se intenta mover es realmente un archivo subido mediante HTTP POST y no un ataque de inclusión de archivos locales.

Etapas de Registro: Solo si los archivos se mueven correctamente al servidor, se ejecuta la consulta INSERT en MariaDB. Esto garantiza que no existan registros en la base de datos que apunten a archivos inexistentes.

10.5. Ejemplo del Flujo de Nombres

Concepto	Valor de Entrada	Valor Procesado
Canción	"Zafar"	Zafar
Artista	"La Vela Puerca"	La_Vela_Puerca
Timestamp	1711834200	1711834200

Resultado final en el Servidor:

uploads/canciones/La_Vela_Puerca/Zafar_(voz)_1711834200.mp3

uploads/canciones/La_Vela_Puerca/Zafar_(instrumental)_1711834200.mp3

uploads/letras/La_Vela_Puerca/Zafar_1711834200.lrc

Ejemplo de como se guardará las carpetas para las canciones y letras

```
// limpiamos los nombres (Evitamos espacios y caracteres raros en carpetas y archivos)
// "Rosalia - Motomami" -> "Rosalia_Motomami"
$artista_folder = str_replace(' ', '_', preg_replace('/[^A-Za-z0-9\-\ ]/', '', $artista));
$nombre_limpio = str_replace(' ', '_', preg_replace('/[^A-Za-z0-9\-\ ]/', '', $titulo));
```

Ejemplo de como se guardan los archivos de instrumental, voz y letra

```
//Creamos el nombre final (Título_Tiempo.extension)
// Añadimos time() al final por si subes dos veces la misma canción, que no se borren
$nombre_final_voz = $nombre_limpio . "_(" . date('s') . ".") . $ext_voz;
$nombre_final_instrumental = $nombre_limpio . "_(" . date('s') . ".") . $ext_instrumental;
$nombre_final_letra = $nombre_limpio . "_(" . date('s') . ".") . $ext_letra;
```

Ejemplo de la creación de carpetas

```
//creamos las carpetas si no existen (con permisos totales para evitar problemas al subir archivos)
// El 0777 es permisos totales, el 'true' permite crear carpetas anidadas
if (!is_dir($dir_canciones)) {
    mkdir($dir_canciones, 0777, true);
}
if (!is_dir($dir_letras)) {
    mkdir($dir_letras, 0777, true);
}
```

11. Motor de Ejecución y Gestión de Sesión

Este apartado describe la lógica central de "Kantabile", que gestiona la interacción en tiempo real entre el usuario, la base de datos y los flujos multimedia sincronizados.

11.1. Gestión Dinámica de Colas Privadas

El sistema implementa una arquitectura de cola personalizada mediante el uso de sesiones de PHP y consultas SQL filtradas por identificador de usuario (\$idUsuario).

Aislamiento de Sesión: Mediante la instrucción WHERE id_usuario = '\$idUsuario', el sistema garantiza que la experiencia sea multi-instancia, permitiendo que cada usuario gestione su propia lista de reproducción sin interferencias.

Gestión de Turnos: El sistema identifica automáticamente la canción activa mediante una consulta con LIMIT 1 y ordenación ascendente por ID, asegurando que el flujo de la sesión siga el orden cronológico de selección.

Control de Estado (Mover/Quitar): Se han desarrollado scripts de soporte (**cancion_mover.php**, **cancion_quitar.php**) que permiten al usuario interactuar con la cola en tiempo real (añadir mediante **fetch** sin recargar, quitar o reordenar). En el caso

de reordenar (subir/bajar), el sistema realiza un intercambio de identificadores en la base de datos utilizando temporalmente el ID = 0 para evitar conflictos de clave única.

11.2. Arquitectura del Reproductor Dual

Una de las innovaciones técnicas del proyecto es el uso de un sistema de audio sincronizado por software.

Dualidad de Pistas: El reproductor carga simultáneamente dos nodos de audio HTML5: la pista de Voz (guía) y la pista Instrumental.

Sincronización de Flujos: Mediante JavaScript, se ha vinculado el `currentTime` de ambas pistas. Al pausar, reproducir o desplazar la barra de progreso (evento `onseeking`), los dos archivos se mantienen en fase con una precisión de milisegundos.

Modo Guía Dinámico: Se implementó una función de mutado selectivo que permite al usuario alternar entre la versión con voz y la instrumental sin interrumpir la reproducción, facilitando el aprendizaje de la canción.

Automatización de Intro y Cuenta Atrás: Al iniciar una canción, el sistema intercepta el evento `onplay` para pausar los flujos de audio temporalmente y desplegar una pantalla de presentación. Un temporizador en JavaScript gestiona una cuenta atrás visual de 5 segundos (destacando en rojo los últimos 3 segundos con un aviso de "¡PREPÁRATE!") antes de liberar de forma síncrona ambas pistas desde el segundo cero.

11.3. Motor de Sincronización Avanzada

El sistema utiliza un algoritmo de procesamiento en el lado del cliente para transformar archivos de texto `.lrc` en una experiencia visual interactiva.

Parsing de Micro-tiempos: A diferencia de reproductores básicos que solo detectan líneas, este motor utiliza Expresiones Regulares (RegExp) para detectar marcas de tiempo de frase [...] y marcas de palabra <...>. Esto permite un resaltado progresivo (tipo "efecto de llenado/barrido progresivo de color") dentro de la misma frase.

Ciclo de Renderizado: En lugar de intervalos fijos, se utiliza el método `requestAnimationFrame`. Esto permite que el "engine" de sincronización se ejecute a la tasa de refresco del monitor (normalmente 60fps), garantizando una transición suave en las barras de progreso de cada palabra mediante variables CSS.

Lógica de Pre-visualización: El motor calcula dinámicamente el índice de la frase actual y la siguiente, permitiendo al usuario anticipar la letra que debe cantar (campo línea-siguiente).

11.4. Interfaz de Escenario

Para profesionalizar la experiencia de usuario, se ha desarrollado un módulo de Escenario Externo.

Comunicación Inter-Ventana: Utilizando el objeto `window.open`, el sistema abre una ventana secundaria independiente del panel de control.

Sincronización del DOM: Mediante un puente de datos en JavaScript, la ventana principal inyecta el contenido procesado de las letras en la ventana del escenario en tiempo real. Esto permite que el monitor principal sea utilizado para la gestión (cola/búsqueda) mientras el monitor secundario muestra exclusivamente la letra al cantante, simulando un entorno de karaoke profesional.

12. Navegación y Flujo de Trabajo

En este apartado se describe la arquitectura de navegación de la aplicación, diseñada bajo un enfoque de experiencia de usuario (**UX**) simplificada para evitar distracciones en entornos de ocio (karaoke).

12.1. Mapa del Sitio

La aplicación se estructura en cuatro nodos principales interconectados:

- **Index / Login:** Puerta de entrada donde se gestiona la autenticación.
- **Dashboard (Biblioteca):** El centro neurálgico donde el usuario busca canciones y gestiona su perfil.
- **Reproductor de Karaoke:** Interfaz de ejecución donde se procesa el audio y la letra.
- **Ventana de Escenario:** Una extensión visual independiente para la proyección de la letra.

12.2. Descripción de los Flujos Principales

Flujo de Selección de Canción:

Para que un usuario pueda cantar, el sistema sigue este camino lógico:

Búsqueda: El usuario utiliza el buscador dinámico. El sistema filtra en tiempo real sobre la base de datos.

Registro en Cola: Al hacer clic en "Añadir a la cola", se dispara una petición POST que registra el `id_cancion` y el alias del cantante.

Carga de Assets: Cuando llega el turno, el sistema carga simultáneamente tres elementos: el audio guía, el audio instrumental y el archivo de texto `.lrc`.

Flujo de Control de Visualización:

Una de las innovaciones de "Kantabile" es su flujo de visualización dual, pensado para monitores externos o proyectores:

Acción del Usuario: Desde el reproductor, el usuario pulsa "Modo Escenario".

Apertura de Nodo: JavaScript abre una ventana pop-up limpia (sin controles, solo letra).

Sincronización: El flujo de datos de la ventana principal se "espeja" en la secundaria. Esto permite que el gestor controle el volumen y la cola mientras el cantante solo ve la letra.

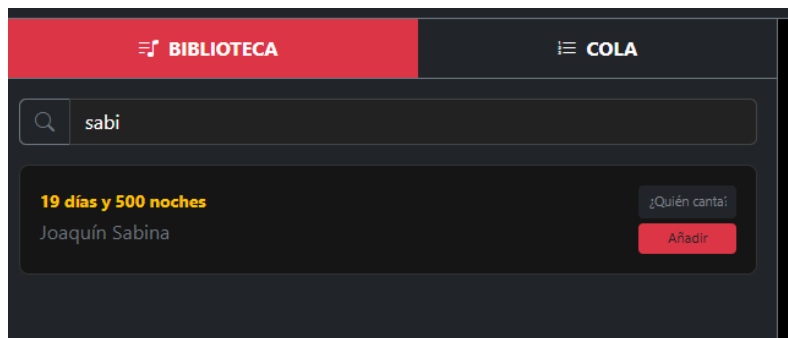
12.3. Diseño de la Interfaz

La navegación se ha diseñado para no ser intrusiva.

Sidebar Fijo: El menú lateral permite saltar entre la biblioteca y la cola de espera sin detener la música que esté sonando.

Feedback Visual: Se han implementado estados de "Cargando" y notificaciones de éxito cuando una canción se añade correctamente a la lista.

Un ejemplo visual de la interfaz sería el que veremos a continuación

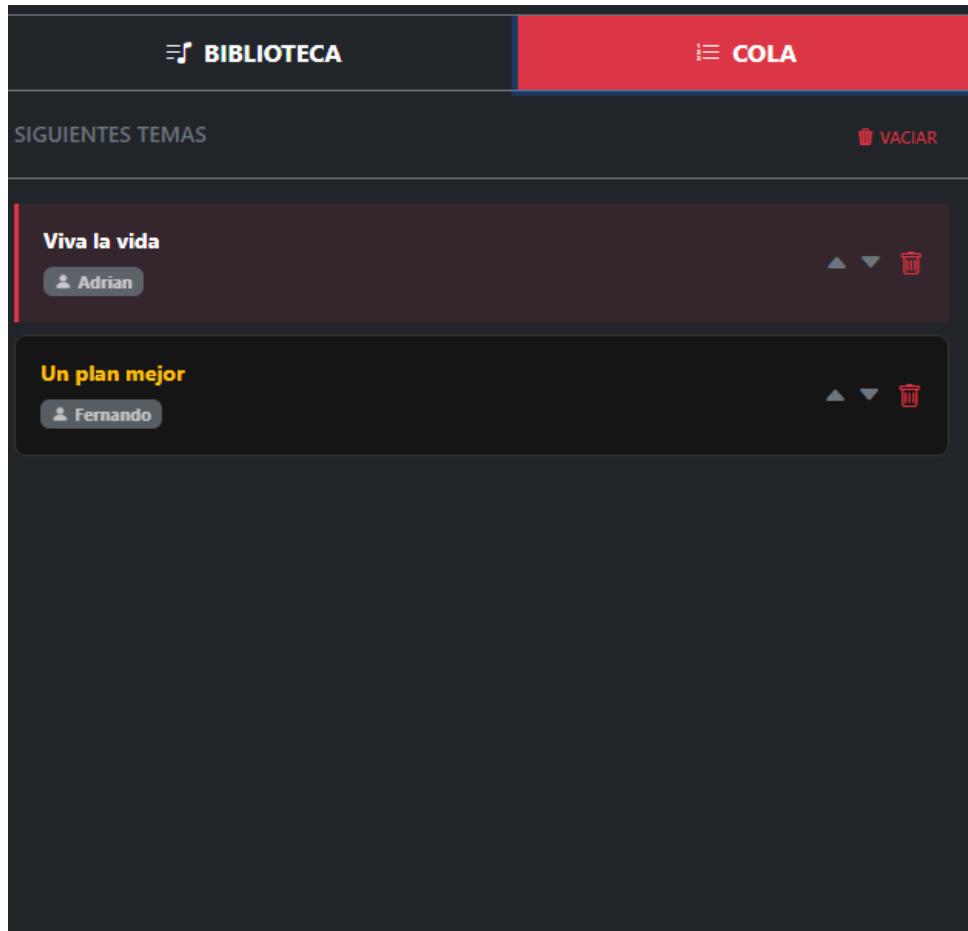
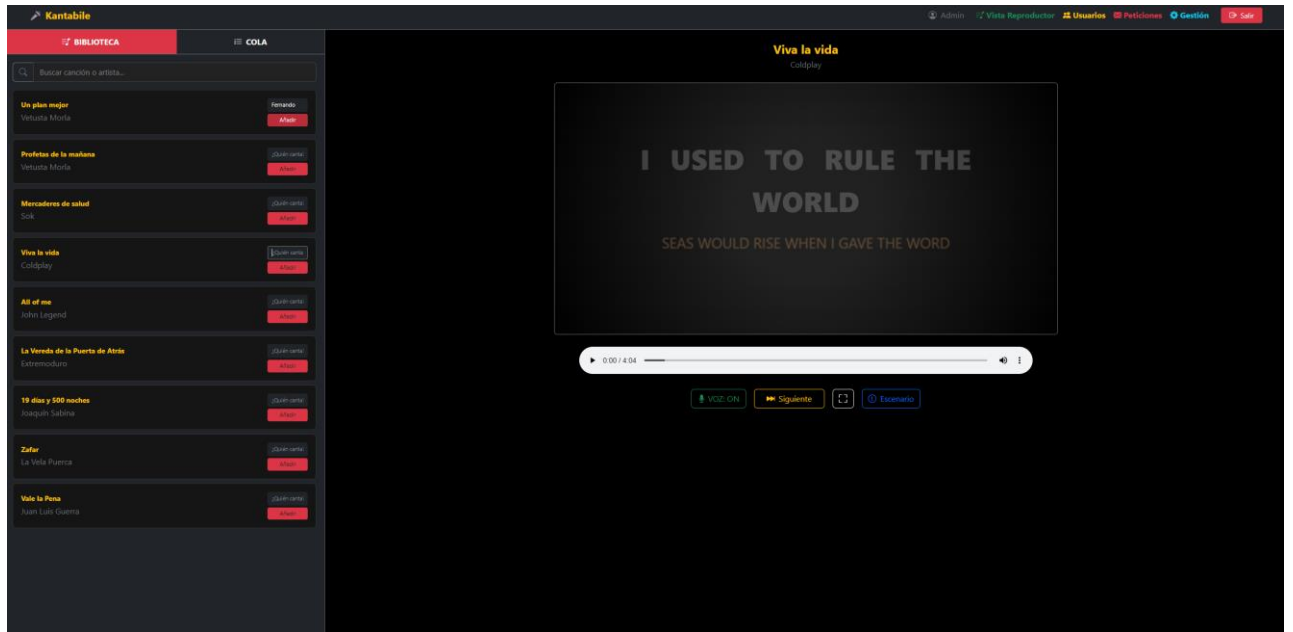


El buscador dinámico en esta imagen busca coincidencias basándose en artista, canción o estilo.

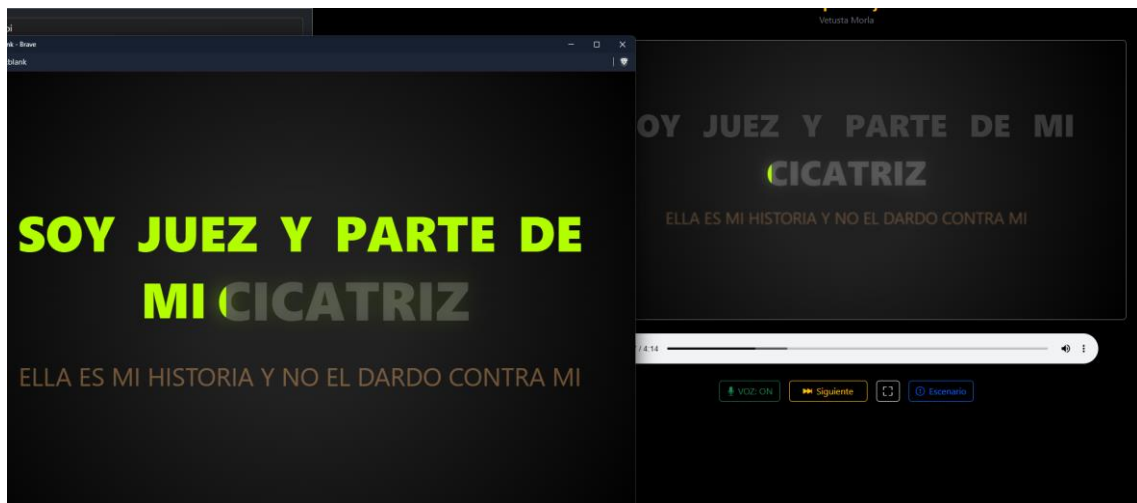
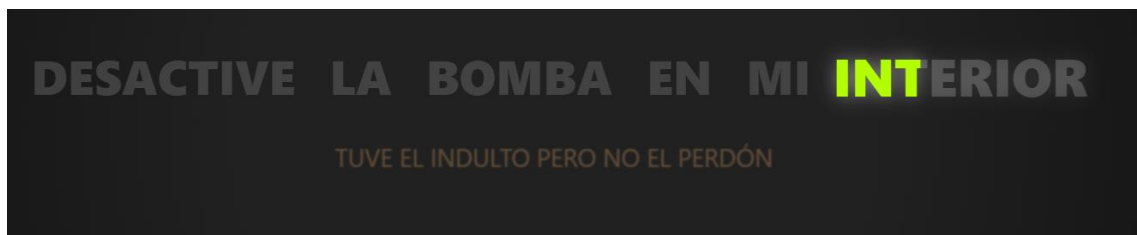
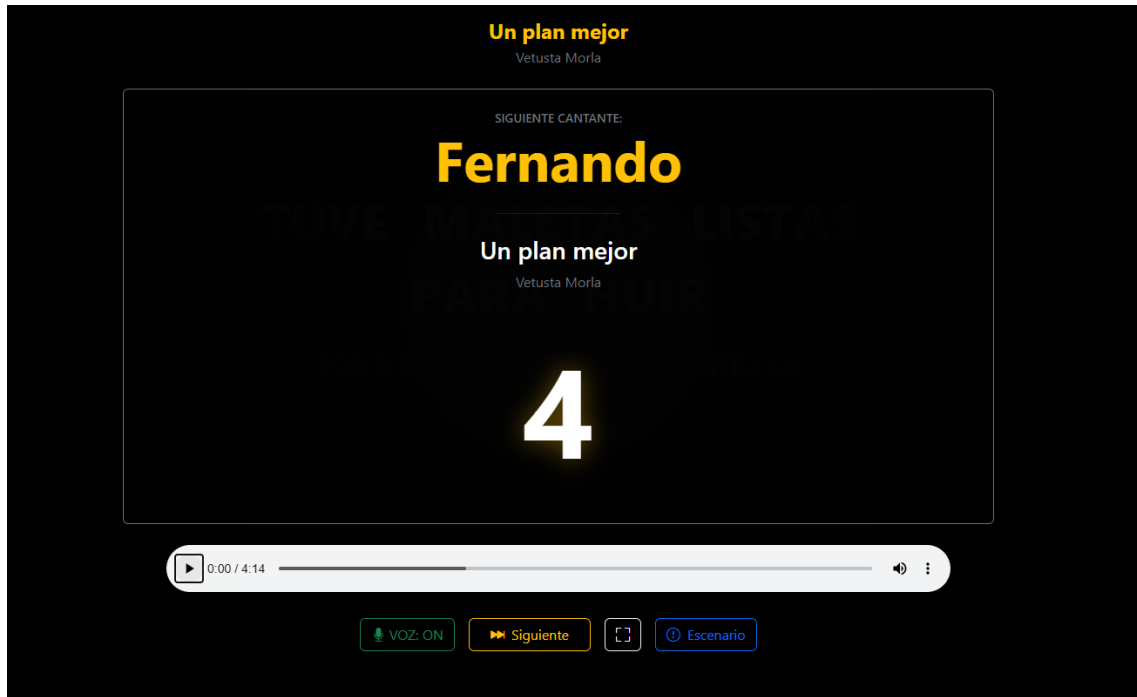
En la siguiente imagen se observará la interfaz completa de una sola página donde se ve mediante tabs tanto la biblioteca con un buscador y un input para el alias como la gestión de colas de reproducción.

A su vez en la derecha tenemos el reproductor, que es el que hace la magia de este proyecto.

Proyecto Intermodular-Operación kantabile



Proyecto Intermodular-Operación kantabile



13. Conectividad e Infraestructura de Comunicación

La robustez de "Kantabile" reside en la comunicación eficiente entre sus distintos componentes. A continuación, se describen los niveles de conectividad implementados:

13.1. Comunicación Cliente-Servidor

La interacción entre el front-end (navegador) y el back-end (PHP) se basa en el protocolo HTTP. Se han utilizado peticiones asíncronas para las siguientes tareas:

- ✓ Validación de credenciales.
- ✓ Consultas a la base de datos de canciones.
- ✓ Actualización de la cola de reproducción en tiempo real.

13.2. Gestión de Recursos Multimedia

La conectividad con el sistema de archivos del servidor es crítica para la reproducción. El sistema gestiona la carga de:

Flujos de Audio: Los archivos **.mp3** se cargan de forma progresiva mediante el elemento **<audio>** de HTML5.

Metadatos LRC: El archivo de letras se descarga como un recurso estático que el motor de JavaScript procesa localmente para garantizar una sincronización de 0 milisegundos de latencia.

13.3. Conectividad entre Contextos

Una característica técnica destacada es la conectividad entre la ventana principal y la ventana del escenario.

Protocolo de Memoria: No se utiliza una conexión de red para este proceso, sino una comunicación directa a través del DOM del navegador.

Sincronismo: El objeto `window.open` permite que la ventana maestra actúe como servidor de contenidos y la ventana esclava como receptor de renderizado, asegurando que la letra se vea exactamente igual en ambas pantallas sin desfase.

13.4. Automatización de Infraestructura y Despliegue Continuo

Una vez que la aplicación web funciona y las distintas pantallas se comunican entre sí, el siguiente reto ha sido garantizar que el sistema sea fácil de mantener, seguro y que esté siempre actualizado sin necesidad de hacer trabajos manuales repetitivos.

En los proyectos web tradicionales, cuando los desarrolladores hacen un cambio en el código, tienen que subir los archivos modificados uno a uno al servidor de internet utilizando programas de envío manual (como los clientes FTP). Este método antiguo

tiene dos grandes problemas para cualquier organización: por un lado, es muy fácil cometer un error humano y olvidar subir un archivo, lo que rompería la página; y por otro lado, mientras se copian los archivos, la web puede dejar de funcionar temporalmente para los usuarios.

Para evitar estos fallos y trabajar como se hace actualmente en las empresas tecnológicas, hemos decidido crear un sistema de **automatización del mantenimiento**.

Esto significa que el servidor se cuida y se actualiza de forma autónoma gracias a dos herramientas:

Un sistema de **Actualización Continua (mediante GitHub Actions)**: Cada vez que los programadores guardan una versión terminada en su repositorio de trabajo, el servidor de internet detecta el cambio, descarga las novedades de forma automática en pocos segundos y las aplica sin que la web deje de estar disponible.

Un sistema de **Mantenimiento de Permisos (mediante Cron Jobs)**: Una tarea invisible que se ejecuta sola en el servidor cada cierto tiempo para revisar que las carpetas de las canciones y los archivos multimedia tengan los permisos correctos. Esto evita que la aplicación falle cuando un usuario intenta subir música nueva.

Para lograr que el servidor se actualice solo cada vez que guardamos el trabajo, no hace falta instalar programas complejos en el servidor. Todo se gestiona creando un canal de comunicación seguro entre nuestro almacén de código (GitHub) y el ordenador donde está alojada la web (el servidor de producción).

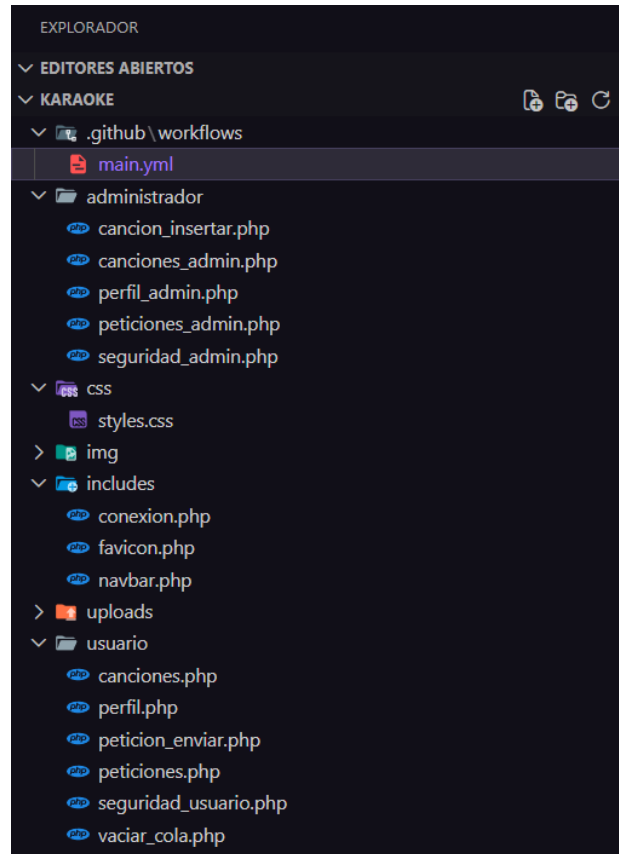
El proceso que hemos seguido para ponerlo en marcha se resume en tres pasos:

13.4.1 Crear archivo de instrucciones.

Lo primero que se hace es crear una carpeta especial en la raíz de nuestro proyecto llamada `.github` y, dentro de ella, otra llamada `workflows`. Ahí dentro es donde guardamos un archivo de texto con extensión `.yml` (normalmente llamado `main.yml`).

Este archivo no contiene código de la página web (no es PHP ni JavaScript), sino que es una **lista de tareas ordenadas** escritas en un formato muy limpio que GitHub sabe leer. Es, literalmente, el manual de instrucciones que seguirá GitHub de forma automática.

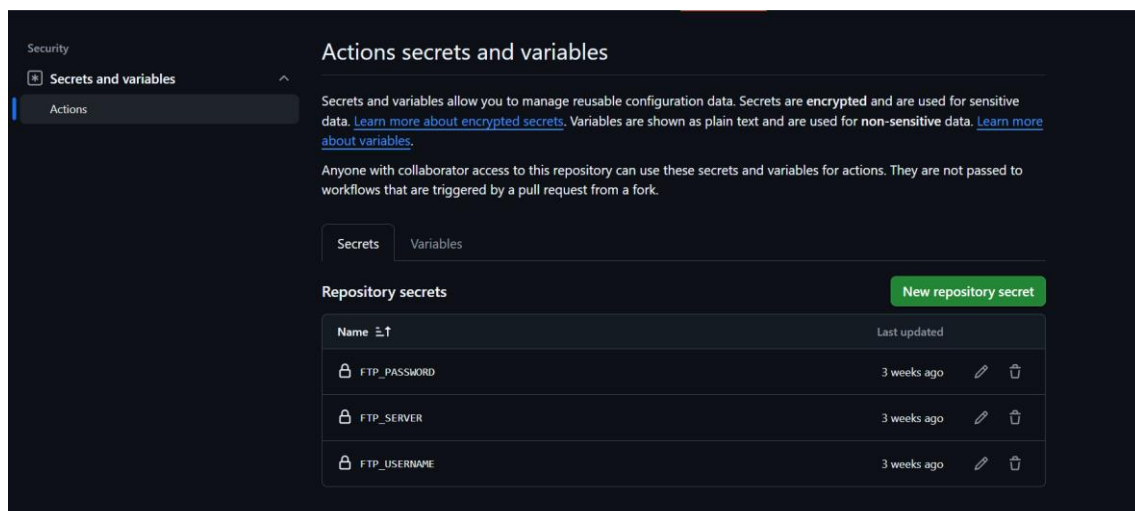
Proyecto Intermodular-Operación kantabile



13.4.2 Guardar las contraseñas en secreto

Como el servidor requiere una contraseña o una llave de seguridad para dejar entrar a alguien a modificar los archivos, no podemos escribir esa contraseña directamente en el archivo main.yml (porque cualquiera que mirase nuestro código podría verla).

Para solucionar esto, hemos usado una opción de GitHub llamada Secrets (Secretos). Hemos guardado de forma oculta los datos de conexión de nuestro servidor (como la dirección IP, el usuario y la contraseña o llave SSH). El archivo main.yml sabe que tiene que usar esos datos secretos, pero nadie los puede leer desde fuera.

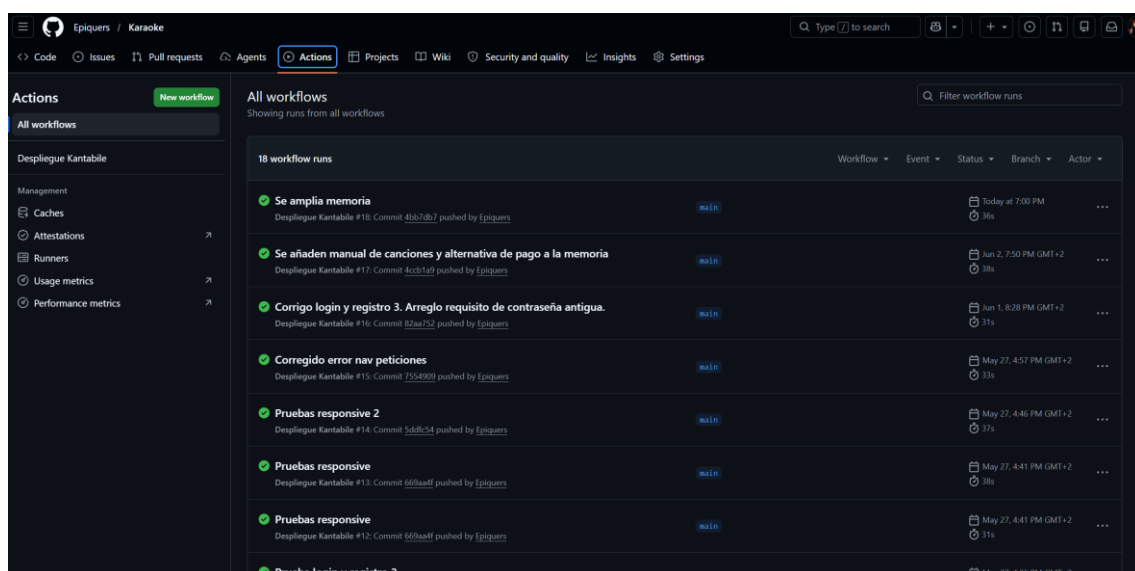


Como se puede observar en la captura de pantalla posterior, la pestaña **"Actions"** de nuestro repositorio de GitHub actúa como un diario de abord de mantenimiento del servidor. Cada fila representa un momento en el que hemos guardado y subido cambios de nuestro código.

El símbolo del círculo verde con el **'tick'** es la confirmación visual de que todo el proceso automático ha funcionado correctamente. Al ver ese indicador, podemos comprobar que:

- ✓ GitHub detectó nuestro cambio en la rama principal (**main**).
- ✓ Se conectó de forma segura con el servidor de producción del proyecto.
- ✓ Se descargaron las actualizaciones con éxito en el servidor y la web se refrescó en milisegundos.

Si en algún momento hubiéramos cometido un error de sintaxis o de conexión, el círculo se volvería de color rojo, avisándonos de inmediato y deteniendo el proceso para proteger la página web y evitar que se rompa de cara al público.



13.4.3 Funcionamiento y programación del archivo main.yml

Al procesar este archivo, el sistema lee directamente las tareas (*jobs*) y pasos (*steps*) que tenemos programados, los cuales se dividen en tres órdenes muy sencillas:

- **El entorno de trabajo (runs-on: ubuntu-latest):** Le pedimos a GitHub que nos "preste" un servidor virtualizado basado en Linux durante unos segundos para procesar el entorno y preparar el envío de los archivos de forma totalmente aislada en la nube.
- **La obtención del código (uses: actions/checkout@v3):** Es el primer paso operativo (*step*) del despliegue. Le ordena al servidor virtual que descargue y prepare una copia exacta del código actual de nuestro repositorio para poder operar con él.

- **La conexión y transferencia por FTP** (uses: SamKirkland/FTP-Deploy-Action): El servidor virtual se conecta a nuestro hosting real utilizando las credenciales encriptadas en GitHub (secrets.FTP_SERVER, FTP_USERNAME y FTP_PASSWORD), accede a la ruta de destino (public_html/kantabile/) y se encarga de subir únicamente los archivos modificados. Además, gracias a la directiva delete-removed: false, nos aseguramos de que no se borre ningún archivo que exista solo en el servidor (como configuraciones locales o subidas de usuarios).

```

main.yml x
.github > workflows > main.yml
7 jobs:
8   web-deploy:
9     name: 🚀 Desplegando a cPanel
10    runs-on: ubuntu-latest
11    steps:
12      - name: 📄 Obteniendo el código actual
13        uses: actions/checkout@v3
14
15      - name: 📁 Subiendo archivos por FTP
16        uses: SamKirkland/FTP-Deploy-Action@v4.3.4
17        with:
18          server: ${ secrets.FTP_SERVER }
19          username: ${ secrets.FTP_USERNAME }
20          password: ${ secrets.FTP_PASSWORD }
21          server-dir: public_html/kantabile/
22          delete-removed: false

```

13.5. Tareas Programadas y Automatización de Permisos en el Servidor

Para terminar de asegurar que la aplicación funcione siempre de forma autónoma, el último paso ha sido configurar un sistema de **mantenimiento invisible** dentro del propio servidor de internet. Esto se consigue utilizando una herramienta del sistema operativo **Linux** llamada **Cron Job** (que se traduce como "**tarea programada**").

13.5.1. El problema de los permisos en un proyecto vivo

En una aplicación web como nuestro proyecto "**Kantabile**", las carpetas del servidor no son todas iguales:

Por un lado, tenemos las carpetas donde están nuestros archivos de código (los **PHP** y **JavaScript**). Por seguridad, estos archivos deben estar protegidos para que nadie desde fuera los pueda modificar o manipular.

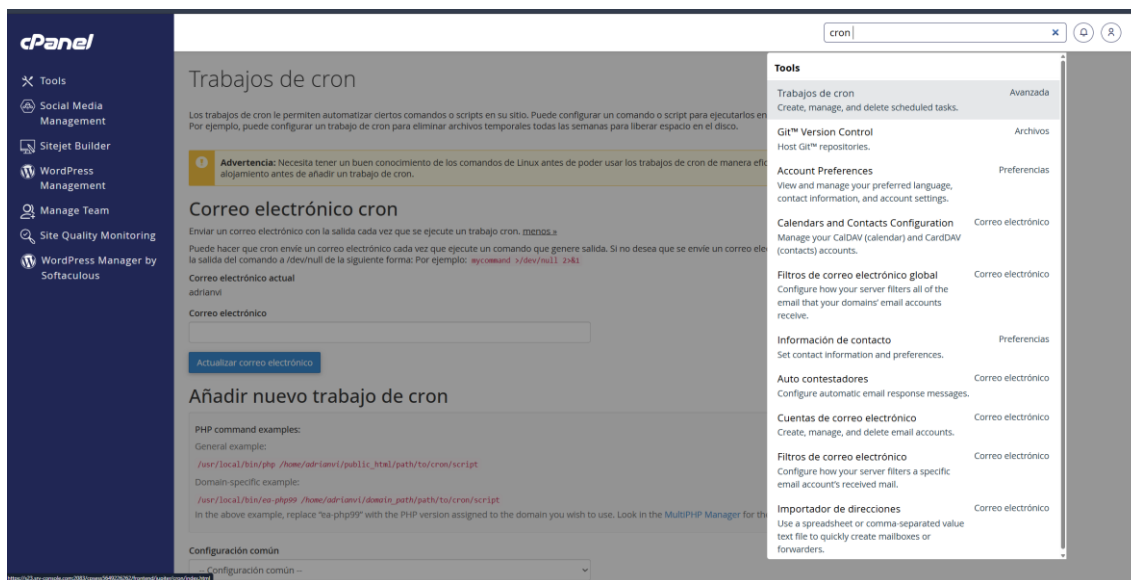
Por otro lado, tenemos las carpetas especiales (como la de las canciones subidas o los archivos de texto **.Irc**). Estas carpetas necesitan obligatoriamente tener el permiso de escritura activado, porque si un administrador intenta subir una nueva canción y la carpeta está bloqueada, la web dará un error de "Permiso Denegado" y la subida fallará.

El problema real ocurre cuando actualizamos la web con **GitHub Actions** (el punto anterior). Al descargarse los archivos nuevos automáticamente, a veces los permisos de las carpetas se desconfiguran o se vuelven demasiado estrictos por defecto.

13.5.2. La solución: Un revisor automático paso a paso

En lugar de tener que entrar nosotros a mano en la consola del servidor cada vez que hay un problema a cambiar los permisos, hemos programado una tarea para que el servidor lo haga solo. El proceso funciona de la siguiente manera:

- ✓ **El reloj del servidor (El "Crontab"):** Hemos accedido al planificador interno del servidor y le hemos puesto una regla de tiempo fija (por ejemplo, que se ejecute de forma automática una vez al día de madrugada, o cada hora).
- ✓ **La orden de propiedad (chown):** Al llegar la hora señalada, el servidor ejecuta una orden interna que asegura que todas las carpetas multimedia pertenezcan al usuario del servidor web. Así nos aseguramos de que la aplicación web siempre tenga el control total sobre esos archivos.
- ✓ **La orden de apertura (chmod):** Acto seguido, el sistema aplica los permisos exactos: cierra con candado los archivos de código para que sean seguros, y "abre" las carpetas de almacenamiento de música para que se puedan seguir añadiendo canciones sin bloquearse nunca.



The screenshot shows the cPanel 'Cron Jobs' (Trabajos de cron) interface. It includes a sidebar with navigation links like Tools, Social Media Management, Sitejet Builder, WordPress Management, Manage Team, Site Quality Monitoring, and WordPress Manager by Softaculous. The main content area has a warning about Linux commands, a section for 'Correo electrónico cron' (Cron email), and a section for 'Añadir nuevo trabajo de cron' (Add new cron job) with PHP command examples. A right-hand sidebar lists various tools like Git Version Control, Account Preferences, Calendars and Contacts Configuration, etc.

Trabajos de cron actuales

Minuto	Hora	Día	Mes	Día de la semana	Comando	Acciones
0	0	*	*	*	<code>cd /home/adrianvi/public_html; /bin/nice -n15 /usr/bin/php -q wp-cron.php >/dev/null 2>&1</code>	Editar Eliminar
0	0	*	*	*	<code>find /home/adrianvi/kantabile.adrianvincent.es -type d -exec chmod 755 {} \; && find /home/adrianvi/kantabile.adrianvincent.es -type f -exec chmod 644 {} \;</code>	Editar Eliminar

Como se muestra en las capturas de pantalla se puede comprobar las líneas de comandos exactas que hemos dejado programadas en el servidor de producción.

Con esta configuración, aunque un desarrollador altere un permiso por error o una actualización automática de GitHub los modifique, el servidor corregirá el fallo por sí solo en su próximo ciclo de ejecución, el cual en nuestro caso lo dejamos en 24 si la actividad de desarrollo no es muy frecuente si no, se puede editar y bajar o subir el tiempo y de esa manera si quieres que el cambio se haga más rápido por necesidad pues se pone cada minuto y al siguiente minuto después de guardar se ejecutaría el comando, garantizando que los usuarios puedan usar el karaoke las 24 horas del día sin interrupciones por problemas de almacenamiento.

Desglosando un poco ambos comandos y resumiendo para mayor brevedad es que, el primer comando lo que hace es acceder a la carpeta principal y ejecutar el archivo de mantenimiento, ósea el cron job a baja prioridad de manera que no requiere apenas recursos mientras se utiliza la web, y así se garantiza un mejor funcionamiento de esta y evita que la pagina vaya lenta o se bloquee, al final de ese comando se anulan los avisos al servidor, para que no acumule archivos basura de avisos por ejecución.

Y por último el segundo comando es el que reconfigura y sanea todos los permisos del proyecto automáticamente, el cual se divide en dos bloques y el segundo no se ejecuta hasta que el primero termine o este todo correcto, de lo contrario, simplemente no se ejecuta.

Y la primera parte del comando lo que hace es acceder o buscar en este caso, en la ruta del subdominio y filtra la búsqueda y selecciona solo las carpetas o directorios e ignora los demás archivos sueltos, y para todo lo que encuentra a su paso con esas características sea carpeta o subcarpeta le aplica un cambio de permisos aplicándole el **755** que otorga todos los permisos al propietario pero a los demás usuarios solo podrán acceder y cargar la web pero no modificar nada estructuralmente.

Para la segunda parte del comando, ahí si nos centramos en los archivos sueltos que se encuentra e ignora directorios, ósea todos los de tipo **file**, ya sea un **.txt** o **.lrc** hace lo mismo que en la primera parte del comando, Aplica a todos esos archivos la máscara de seguridad **644**. El 6 permite que el sistema lea y escriba en sus propios archivos de configuración si lo necesita, mientras que los **44** otorgan permisos de solo lectura al resto del mundo. Esto blindo el código fuente de la aplicación, impidiendo que un usuario externo intente manipular o inyectar código malicioso en los archivos de este proyecto.

14. Manual de incorporación de canciones al sistema de karaoke

Este documento describe el procedimiento completo para añadir una nueva canción al catálogo del sistema de karaoke. El proceso incluye la obtención de la canción original, la generación de las pistas de audio necesarias, la sincronización de la letra y la carga final de todos los archivos en el panel de administración.

Es importante seguir cada uno de los pasos descritos para garantizar que la canción se reproduzca correctamente y que la letra aparezca sincronizada durante la interpretación.

14.1. Obtención de la canción

El primer paso consiste en obtener la canción que se desea incorporar al sistema.

Requisitos del archivo

- La canción debe estar disponible en formato MP3.
- Debe tratarse de una versión completa y de buena calidad.
- Se recomienda utilizar un archivo con una tasa de bits adecuada para garantizar una correcta reproducción.

Verificaciones previas

Antes de continuar con el proceso:

1. Comprobar que el archivo se reproduce correctamente.
2. Verificar que la canción corresponde exactamente a la versión que se desea publicar.
3. Confirmar que el título y el artista son correctos.

Una vez validado el archivo MP3, se puede proceder al procesamiento de audio.

14.2. Procesamiento de audio con Moises

Una vez obtenida la canción original, se utiliza la plataforma **Moises** para separar las distintas pistas de audio necesarias para el funcionamiento del karaoke.

Objetivo

El objetivo de este paso es generar las pistas que utilizará posteriormente el sistema:

- Pista instrumental.
- Pista de voz.
- Canción completa.

Procedimiento

1. Acceder a la plataforma **Moises**.
2. Iniciar sesión con la cuenta correspondiente.
3. Crear un nuevo proyecto o carga de audio.
4. Subir el archivo MP3 de la canción.
5. Esperar a que **Moises** complete el análisis y la separación de pistas.

Descarga de archivos

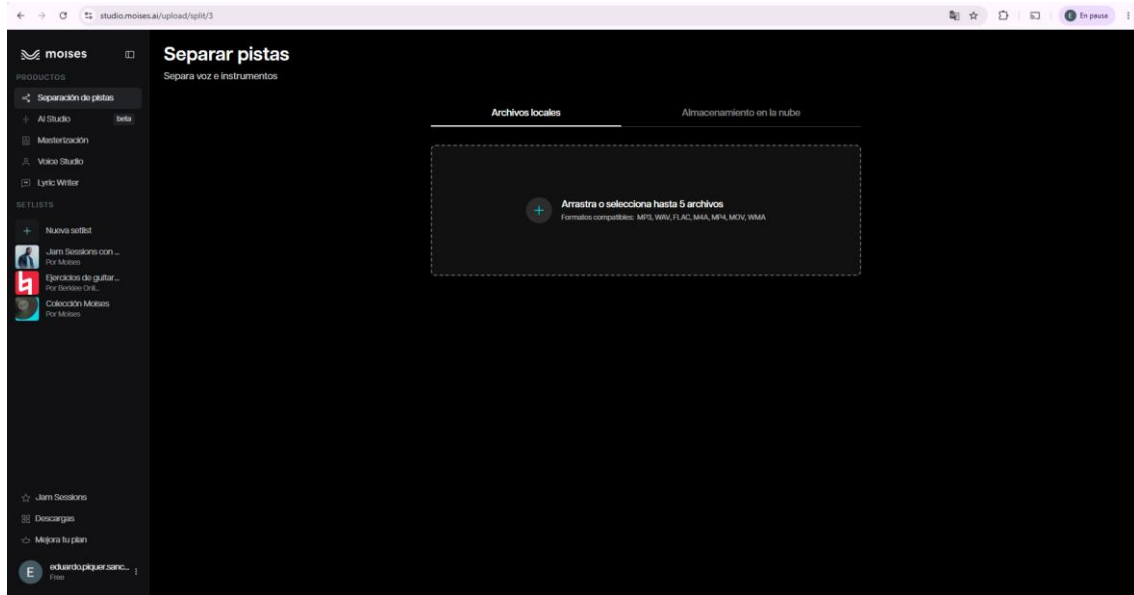
Una vez finalizado el procesamiento, **Moises** generará varios archivos.

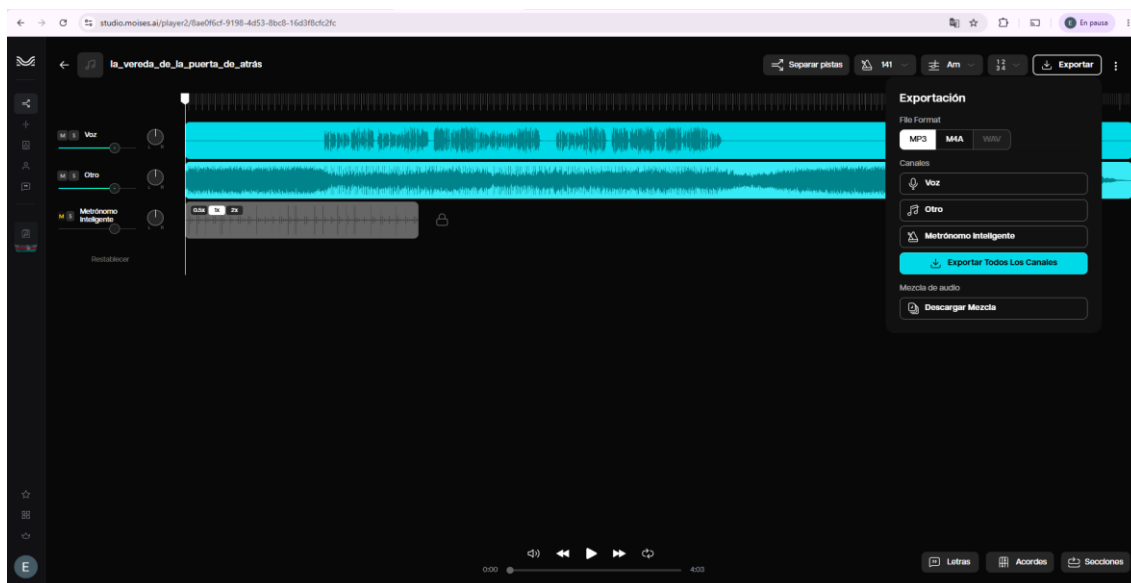
Se deben descargar los archivos generados utilizando siempre el formato estándar configurado en la plataforma.

Los archivos que se utilizarán posteriormente son:

Proyecto Intermodular-Operación kantabile

- Archivo de voz.
- Archivo instrumental.
- Archivo de canción completa.





14.3. Generación y sincronización de la letra

Una vez obtenida la canción completa, es necesario crear el archivo de sincronización de letra que utilizará el reproductor de karaoke.

Para ello se utiliza la herramienta web **QuickLRC**.

Material necesario

Antes de comenzar, deben estar disponibles:

- El archivo de canción completa generado por **Moises**.
- La letra completa de la canción.

Procedimiento

1. Acceder a la herramienta **QuickLRC**.
2. Cargar el archivo de audio correspondiente a la canción completa.
3. Copiar y pegar la letra completa de la canción.
4. Iniciar el proceso de sincronización.

Sincronización manual

La sincronización se realiza manualmente.

Durante la reproducción de la canción, se debe marcar el momento exacto en el que aparece cada palabra de la letra utilizando la letra “t” del teclado o haciendo clic en el botón “Time Next Word”.

El objetivo es conseguir que el texto aparezca en pantalla de forma perfectamente sincronizada con la música.

Recomendaciones

- Escuchar atentamente el inicio de cada frase.
- Corregir posibles desfases durante la revisión.
- Verificar especialmente las transiciones rápidas entre versos.
- Revisar el resultado completo antes de exportar el archivo.

Exportación

Una vez finalizada la sincronización:

1. Guardar el proyecto.
2. Exportar el resultado en formato **.LRC**.

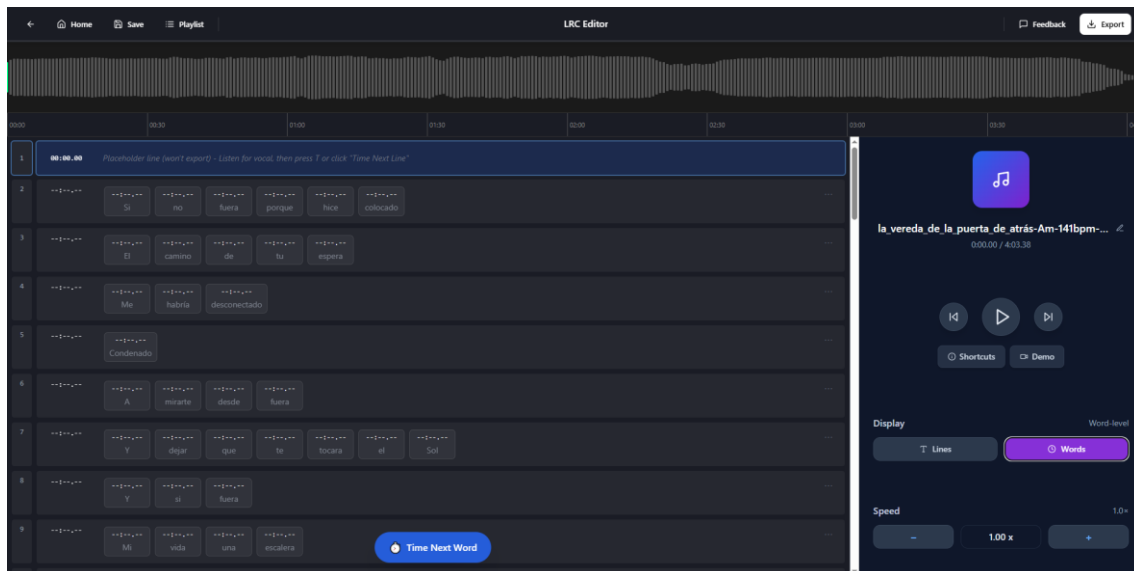
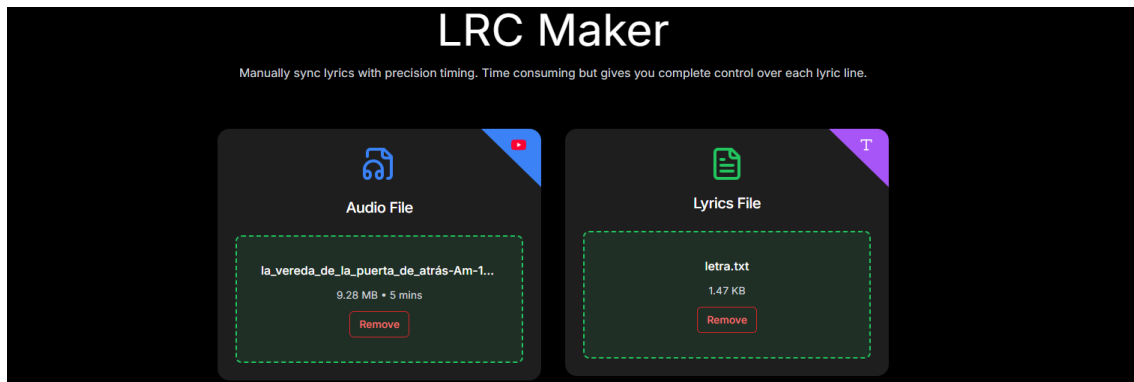
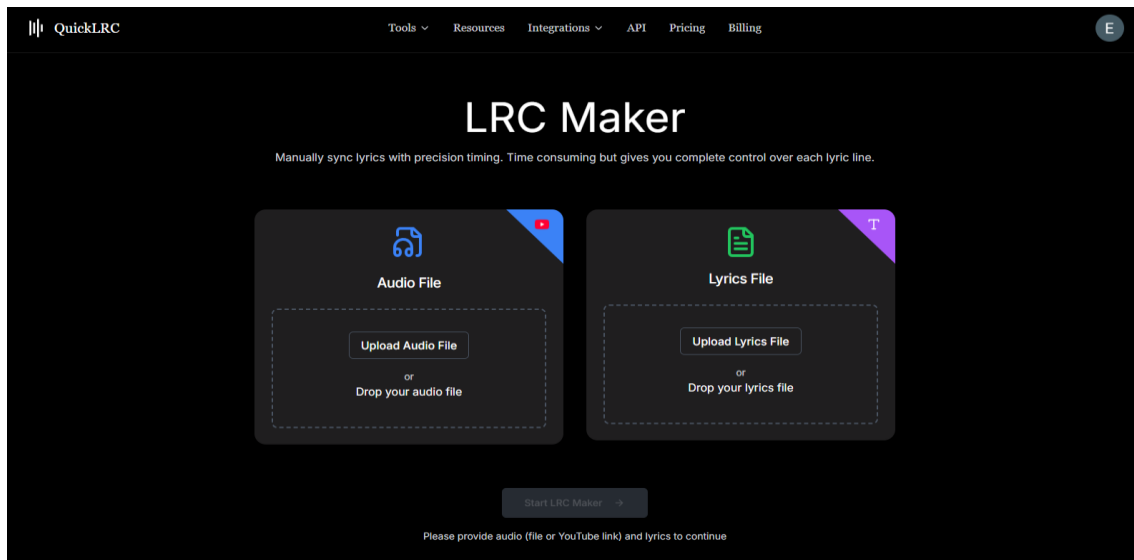
Este archivo contendrá todas las marcas de tiempo necesarias para mostrar la letra sincronizada durante la reproducción de la canción.

Resultado esperado

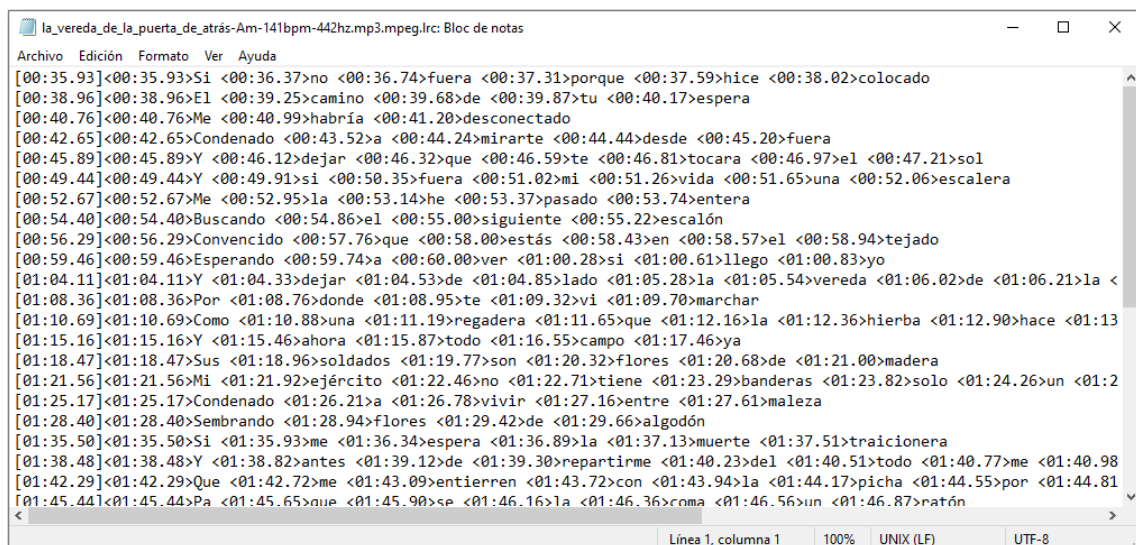
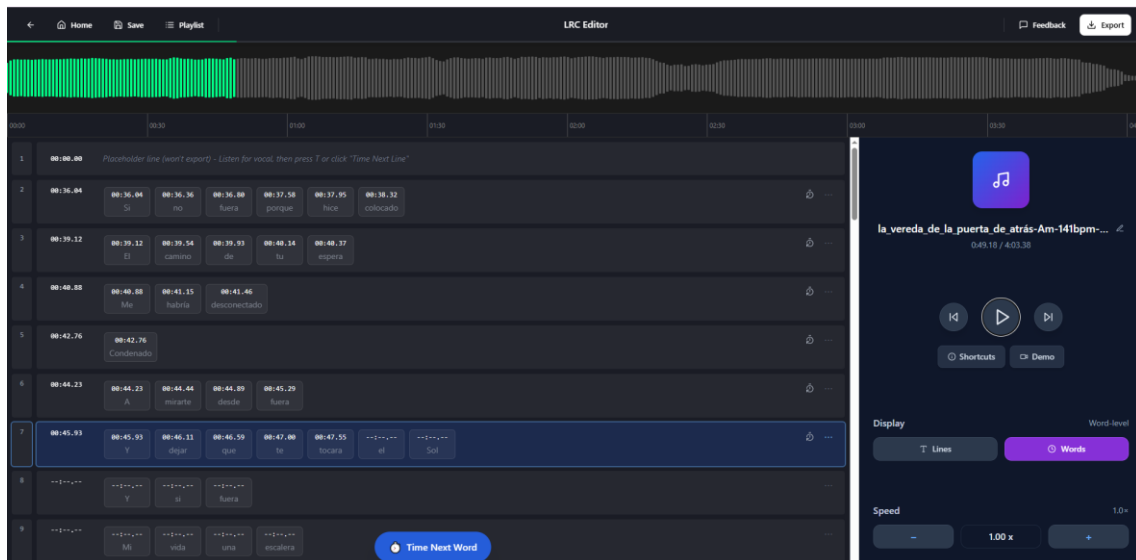
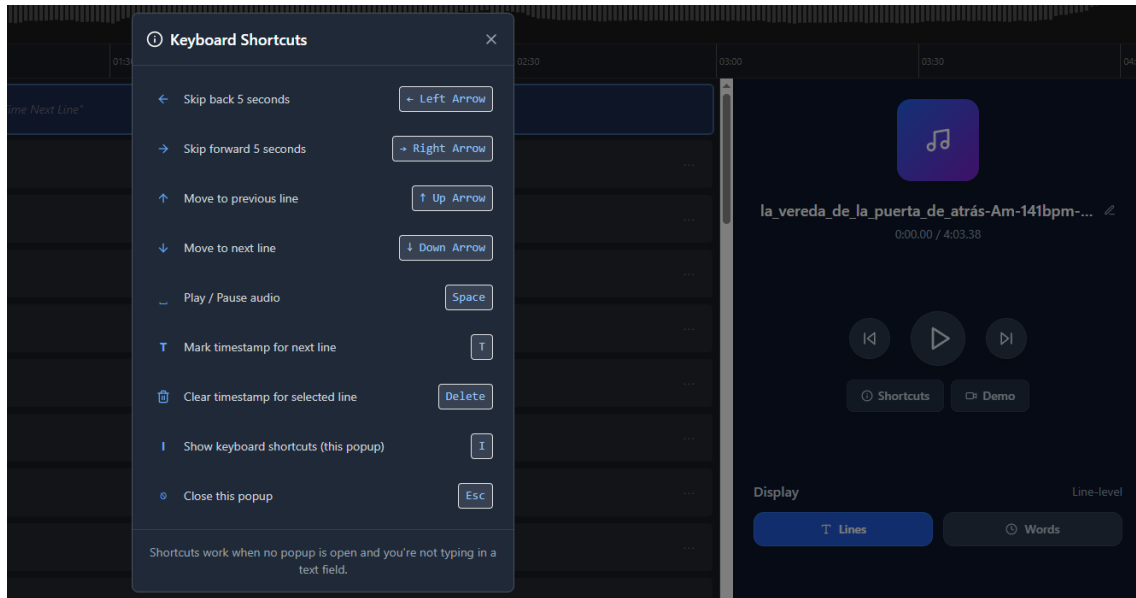
Al finalizar este paso se dispondrá de:

- Archivo de voz.
- Archivo instrumental.
- Archivo de canción completa.
- Archivo de letra sincronizada (.LRC).

Proyecto Intermodular-Operación kantabile



Proyecto Intermodular-Operación kantabile



14.4. Alta de la canción en el sistema de administración

Con todos los archivos preparados, se procede a dar de alta la canción en la plataforma de gestión del karaoke.

Acceso

1. Acceder al panel de administración.
2. Iniciar sesión con un usuario autorizado.
3. Entrar en la sección de gestión de canciones.

Creación de una nueva canción

Seleccionar la opción correspondiente para añadir una nueva canción al catálogo.

Información obligatoria

Introducir los siguientes datos:

Artista

Nombre del artista o grupo musical.

Ejemplos:

- Queen
- Alejandro Sanz
- Mecano

Título

Nombre completo de la canción.

Ejemplos:

- Bohemian Rhapsody
- Corazón Partío
- Hijo de la Luna

Carga de archivos

Adjuntar los archivos generados durante los pasos anteriores:

Archivo de voz

Archivo obtenido desde **Moises** que contiene únicamente la voz de la canción.

Archivo instrumental

Archivo instrumental generado por **Moises** que será utilizado como base musical para el karaoke.

Archivo LRC

Archivo de letra sincronizada generado mediante **QuickLRC**.

Verificación

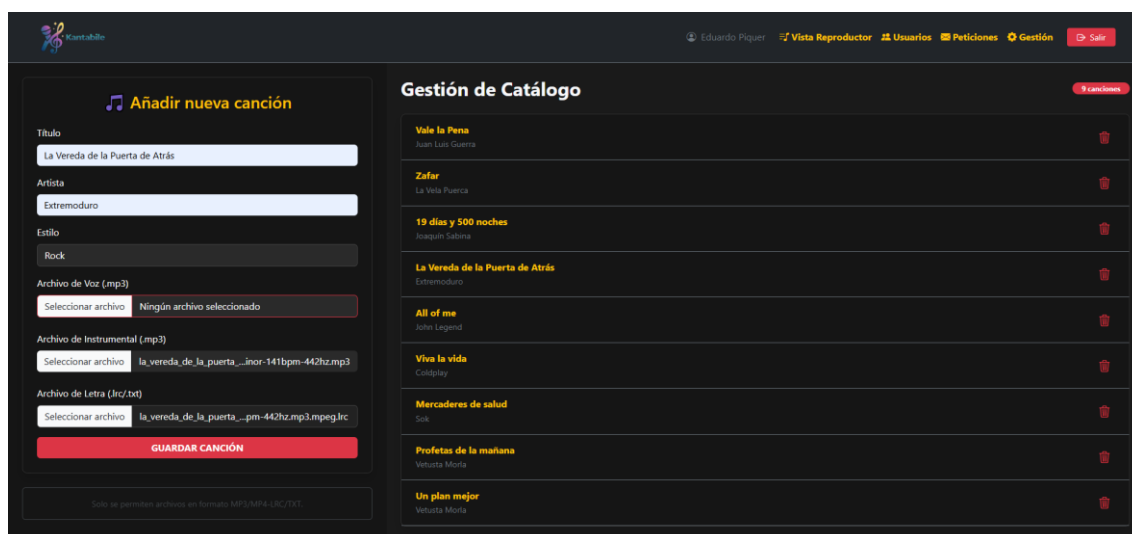
Antes de guardar la canción:

1. Comprobar que los archivos correctos han sido seleccionados.
2. Revisar el nombre del artista.
3. Revisar el título de la canción.
4. Verificar que el archivo LRC corresponde a la misma versión de la canción.

Guardado

Una vez comprobados todos los datos:

1. Pulsar el botón de guardar canción.
2. Esperar a que finalice la carga de archivos.
3. Confirmar que la canción aparece en el listado de canciones disponibles.



The screenshot shows the Kantabile app interface. On the left, the 'Añadir nueva canción' form is visible with fields for Title, Artist, Style, and file uploads for Voice, Instrumental, and Lyrics. The 'Guardar canción' button is at the bottom. On the right, the 'Gestión de Catálogo' section displays a list of songs with their titles, artists, and a delete icon.

Gestión de Catálogo	
Vale la Pena	Juan Luis Guerra
Zafar	La Vela Puerca
19 días y 500 noches	Joaquín Sabina
La Vereda de la Puerta de Atrás	Extremoduro
All of me	José Legrand
Vive la vida	Ciudad
Mercaderes de salud	Sark
Profetas de la mañana	Vetusta Moria
Un plan mejor	Vetusta Moria

14.5. Comprobación final

Tras la creación de la canción, es recomendable realizar una prueba completa.

Validaciones recomendadas

- Verificar que la canción aparece en el catálogo.
- Comprobar que el audio instrumental se reproduce correctamente.
- Verificar que la letra aparece sincronizada.
- Confirmar que no existen errores en el título o en el artista.
- Revisar que la experiencia de karaoke es correcta desde el punto de vista del usuario.

Una vez superadas estas comprobaciones, la canción quedará disponible para todos los usuarios del sistema.

15. Alternativa de pago para generación automática de karaoke: Youka

Youka es una herramienta web diseñada para crear vídeos de karaoke de forma casi automática a partir de un archivo de audio MP3. Su objetivo es simplificar todo el flujo de trabajo habitual (separación de pistas, obtención de letra, sincronización y creación de vídeo final) en un único proceso.

¿Cómo funciona?

El funcionamiento de **Youka** se basa en un sistema automatizado de análisis de audio y sincronización de texto mediante inteligencia artificial.

1. Carga del archivo de audio

El usuario sube un archivo en formato MP3 que contiene la canción original completa.

- El sistema acepta archivos de audio estándar.
- No es necesario realizar ninguna separación previa de pistas.

2. Procesamiento automático del audio

Una vez cargado el archivo, la plataforma realiza varias tareas de forma automática:

- Separación de la voz y la instrumental.
- Análisis de la estructura de la canción (versos, estribillos, pausas).
- Detección de patrones de voz para la sincronización posterior.

Este proceso sustituye herramientas externas como **Moises**, ya que todo se realiza dentro de la misma plataforma.

3. Introducción o detección de la letra

El usuario puede:

- Pegar manualmente la letra de la canción, o
- Utilizar la detección automática (cuando está disponible)

El sistema asocia el texto con el audio para preparar la sincronización.

4. Sincronización automática de la letra

Youka genera una sincronización automática entre la música y la letra.

- Divide la canción en segmentos temporales.
- Asocia cada línea de la letra a su punto exacto en el audio.
- Permite ajustes manuales si algún fragmento no queda perfectamente alineado.

Esto elimina la necesidad de herramientas externas como **QuickLRC**.

5. Generación del vídeo karaoke

Una vez completada la sincronización, la plataforma genera automáticamente un vídeo karaoke en formato MP4.

El vídeo incluye:

- Música instrumental como base sonora.
- Letra sincronizada en pantalla.
- Efectos visuales de resaltado típicos de karaoke (tipo “bounce” o coloreado de palabras).
- Opciones de personalización (tipografía, fondo, estilo visual).

6. Exportación del resultado

El usuario puede exportar el resultado final en distintos formatos, normalmente:

- Vídeo MP4 (karaoke completo listo para reproducir).
- Archivo de proyecto editable (para futuras modificaciones).

Ventajas principales

- No requiere herramientas externas (todo en una sola plataforma).
- Reduce el proceso manual de sincronización de letras.
- Genera resultados listos para uso directo en vídeos o plataformas de karaoke.
- Permite edición posterior para ajustar errores finos.

Conclusión

Youka es una solución adecuada como alternativa de pago cuando se busca automatizar la creación de contenido karaoke, reduciendo significativamente el tiempo de producción y eliminando pasos intermedios como la sincronización manual de letras o la separación de pistas en herramientas externas.

16. Problemas y retos surgidos durante el desarrollo

Durante el desarrollo del proyecto **Kantabile** han surgido diversos problemas técnicos y de diseño que han requerido un proceso de investigación, pruebas y adaptación de las soluciones inicialmente planteadas.

Uno de los principales retos del proyecto ha sido la carga e incorporación de las canciones en el sistema. En una fase inicial, se consideró el uso de herramientas como **Moises**, ya que permite generar vídeos con separación de pistas y letra sincronizada de forma automática. Sin embargo, tras su análisis, se comprobó que el contenido generado no podía descargarse libremente, lo que limitaba su integración directa en la aplicación.

A partir de esta limitación, fue necesario replantear el enfoque y buscar una alternativa viable que permitiese obtener los recursos necesarios de manera manual. En este proceso, **Moises** sí resultó útil para la separación de pistas de audio, permitiendo obtener versiones instrumentales y de voz de las canciones.

No obstante, el principal desafío consistió en la obtención de la letra sincronizada con tiempos. Tras diversas pruebas con diferentes herramientas y enfoques, finalmente se optó por el uso de **QuickLRC**, que permite generar archivos en formato .lrc con marcas de tiempo asociadas a cada fragmento de la letra.

Durante este proceso se realizaron múltiples pruebas combinando distintas herramientas y métodos hasta encontrar un flujo de trabajo estable y funcional que permitiera integrar correctamente las canciones en el sistema de karaoke. Finalmente, esta solución permitió disponer tanto del audio separado como de la letra sincronizada, lo que hizo posible la implementación del reproductor del proyecto **Kantabile**.

Otro de los desafíos más relevantes ha sido la sincronización entre el vídeo y la letra de las canciones. Conseguir que ambos elementos se reprodujeran de forma coordinada ha requerido varias pruebas y ajustes en la lógica del reproductor para asegurar una experiencia fluida.

Durante el desarrollo se probaron distintas aproximaciones, pero no todas ofrecían el resultado esperado en cuanto a precisión y estabilidad durante la reproducción. Por este motivo, se fue iterando sobre la solución hasta conseguir un comportamiento más fiable.

En este proceso, el uso de herramientas de inteligencia artificial resultó de ayuda para plantear y desarrollar una función más completa que facilitara la sincronización entre el tiempo del vídeo y la aparición de la letra en pantalla. Posteriormente, esta solución se adaptó y ajustó manualmente para integrarla correctamente en el sistema y mejorar su funcionamiento.

Finalmente, se consiguió una sincronización adecuada entre vídeo y letra, lo que ha sido una de las partes más importantes del proyecto **Kantabile**.

Asimismo, se ha afrontado la complejidad de permitir la gestión de la cola de reproducción sin necesidad de recargar la página. Aunque inicialmente se había planteado una solución más sencilla, la implementación final ha requerido una mayor complejidad técnica de la prevista para lograr una actualización dinámica del contenido.

Otro de los puntos a destacar, es el reto que supuso el diseño de la interfaz de usuario, buscando un equilibrio entre simplicidad, usabilidad y atractivo visual. El objetivo ha sido crear una interfaz intuitiva que facilitara la navegación y el uso de la aplicación sin sobrecargar al usuario con elementos innecesarios.

Por último, al trabajar en equipo y dar el paso de trasladar todo el proyecto desde nuestros entornos de desarrollo locales en Windows hacia el servidor real de producción, nos topamos con varios desafíos críticos que ponían en riesgo la estabilidad y seguridad de la aplicación. Uno de los primeros problemas que detectamos fue el peligro que suponía depender de subidas manuales tradicionales mediante programas como FileZilla. Este método tradicional (el clásico FTP a mano) siempre introduce el riesgo de cometer errores humanos, como dejarse alguna carpeta atrás, subir archivos corruptos si se sufre un microcorte de internet o machacar datos en caliente de forma accidental. Para solucionar esto radicalmente, decidimos implementar un flujo automatizado con GitHub Actions, logrando que el sistema compare el código de manera matemática y sube únicamente los cambios exactos de forma limpia, asegurando que la web funcione siempre a la primera tras cada actualización.

Esta automatización traía consigo otra problemática importante: para que GitHub se conecte de forma autónoma a nuestro hosting, necesita conocer las contraseñas del servidor. Dejar estas credenciales apuntadas en texto plano dentro de los archivos del proyecto era una vulnerabilidad inasumible, ya que cualquiera que accediera al repositorio podría comprometer todo nuestro panel de administración cPanel. Resolvimos este contratiempo protegiendo el acceso a través de los GitHub Secrets, logrando que las claves viajen completamente encriptadas y ocultas en la memoria del sistema, de modo que ni nosotros mismos podemos verlas en el código fuente.

Por otro lado, el choque entre sistemas operativos también nos causó dolores de cabeza; programar sobre entornos Windows y desplegar en un servidor Linux real provoca que las configuraciones de permisos internos de los archivos se rompan constantemente. En cuanto el servidor Apache detecta un archivo con permisos incoherentes, bloquea el acceso por seguridad y arroja el temido Error 500, tirando la aplicación por completo. Ante este riesgo, configuramos una tarea programada interna en el propio hosting que corre de forma invisible cada medianoche, revisando el árbol del proyecto y reparando los permisos automáticamente para mantener la web estable y disponible sin intervención manual.

Finalmente, tuvimos que lidiar con el impacto en el rendimiento que generan las tareas de limpieza del sistema, como purgar archivos temporales o registros de la base de datos. Si estos procesos pesados se ejecutaran al mismo tiempo que un usuario navega por la web, saturarían los hilos del servidor provocando tirones y retardos. En una aplicación multimedia como nuestro proyecto, esto rompería por completo la experiencia de usuario, por lo que decidimos automatizar y aislar estas tareas para que se ejecuten solas de fondo en horas de poco tráfico y con una prioridad de CPU muy baja,

garantizando así que el servidor reserve toda su potencia para ofrecer un reproductor de karaoke con latencia cero y sin cortes.

A pesar de estas dificultades, todos los problemas han sido abordados y resueltos, permitiendo completar el desarrollo del sistema de forma satisfactoria.

17. Usuarios de prueba

Para facilitar la evaluación y las pruebas de la aplicación, se han habilitado varios usuarios de prueba con diferentes niveles de permisos. Estas cuentas permiten acceder a las distintas funcionalidades del sistema sin necesidad de realizar un registro previo.

Usuarios de administración:

- AVicent@kantabile.com // 1234
- EPiquer@kantabile.com // 1234

Usuario estándar:

- FUrena@kantabile.com // 1234
- VVerdu@kantabile.com // 1234
- Invitado@kantabile.com // 1234

18. Reflexión final

En conclusión, este proyecto ha consistido en el desarrollo de una aplicación web de karaoke responsive, pensada para poder utilizarse tanto en ordenador como en móvil, con la idea de que cualquier persona pueda disfrutar de karaoke en cualquier momento y lugar.

Durante el desarrollo hemos conseguido prácticamente todos los objetivos que nos propusimos al inicio. La aplicación funciona de forma estable y permite reproducir canciones con la letra sincronizada en vídeo, que era una de las partes más importantes del proyecto. Aunque nos habría gustado ir un paso más allá e implementar la posibilidad de añadir cualquier canción de forma directa, esta funcionalidad no ha sido posible por la complejidad técnica que conlleva.

Uno de los mayores retos ha sido la gestión de las canciones y todo el proceso necesario para prepararlas (instrumental, voz, etc.), ya que requiere bastante trabajo previo y no es algo sencillo de automatizar. También ha sido complicado conseguir una buena integración de las letras con el vídeo, ya que es una parte delicada del sistema y ha requerido bastante prueba y error.

Además, el desarrollo del proyecto se ha ido realizando a lo largo del curso, lo que ha hecho que no siempre se pudiera avanzar de forma totalmente continua. La alternancia con otros proyectos, exámenes y posteriormente las prácticas ha provocado que en algunos momentos fuera necesario retomar el trabajo tras periodos sin dedicarle tiempo, y esta falta de continuidad ha afectado en ciertos momentos, ya que a veces costaba retomar el proyecto después de varios días sin trabajarlo.

Aun así, estamos bastante satisfechos con el resultado final. El proyecto funciona correctamente y cumple su objetivo principal, aunque evidentemente siempre hay cosas que se podrían seguir mejorando.

En general, ha sido una experiencia muy positiva, tanto a nivel técnico como de trabajo en equipo, ya que lo hemos desarrollado entre dos personas y hemos tenido que organizarnos y tomar decisiones juntos. Como posible mejora futura, nos gustaría ampliar el proyecto incorporando algún tipo de inteligencia artificial, por ejemplo, para ayudar a generar o sincronizar letras de forma automática, o para facilitar la incorporación de nuevas canciones al sistema.

19. Bibliografía

Para el desarrollo de "**Operación Kantabile**", la resolución de los retos técnicos se ha basado en la consulta constante de las documentaciones oficiales de la industria. Apoyarnos en las fuentes originales nos ha permitido aplicar buenas prácticas de programación, garantizar la seguridad de los datos y cumplir con los estándares web en todo el sistema. En este proceso, el uso de la Inteligencia Artificial ha sido una herramienta estratégica y clave para el equipo. Lejos de usarse para generar código de forma automática, la IA ha actuado como un filtro rápido de documentación y como consultora de viabilidad. Nos ayudó a validar ideas complejas antes de programar como la lectura en tiempo real de archivos **.lrc** sincronizados en milisegundos, permitiéndonos descartar caminos obsoletos, entender mejor las especificaciones de Mozilla y optimizar el tiempo de investigación.

A continuación, se detallan las principales fuentes de información, documentaciones y plataformas consultadas a lo largo de las distintas fases del proyecto:

Documentación Oficial de Tecnologías Core

PHP Manual: Documentación oficial utilizada para la gestión de sesiones seguras (**\$_SESSION**), funciones avanzadas de **arrays** y lógica de control en el **backend**. (<https://www.php.net/docs.php>)

W3Schools: Plataforma de referencia y consulta técnica fundamental utilizada a lo largo de todo el proyecto para validar la sintaxis de etiquetas HTML5, selectores de CSS dinámicos, estructuras de control en JavaScript y ejemplos prácticos de funciones nativas de PHP. (<https://www.w3schools.com>)

W3C (World Wide Web Consortium): Estándares oficiales para maquetación semántica y compatibilidad de navegadores. (<https://www.w3.org>)

Frameworks de Diseño e Interfaz

Bootstrap Documentation: Guía oficial de componentes y el sistema de rejilla (**Grid System**) para lograr que la interfaz con estética **glassmorphism** fuera completamente responsiva y adaptable a cualquier pantalla. (<https://getbootstrap.com/docs>)

DevOps, Despliegue y Automatización

GitHub Actions Documentation: Manuales oficiales de GitHub para la creación de **workflows** en archivos YAML, configuración de tareas en contenedores virtuales (**ubuntu-latest**) y uso de repositorios de código. (<https://docs.github.com/es/actions>)

GitHub Marketplace - FTP Deploy Action: Documentación técnica de la acción de **SamKirkland** utilizada para configurar el despliegue automático del delta de archivos por FTP y el uso de la variable `delete-removed`. (<https://github.com/marketplace/actions/ftp-deploy>)

cPanel Standard Docs: Manuales de administración de servidores para la configuración de directorios web (`public_html`), gestión de subdominios, almacenamiento de bases de datos relacionales y la programación de tareas programadas (Cron Jobs). (<https://docs.cpanel.net>)

GNU/Linux Man Pages: Páginas de manual de Linux consultadas para actualizar y estructurar correctamente los comandos de reparación de permisos (`find`, `chmod`) y la asignación de prioridad de CPU (`nice`). (<https://man7.org/linux/man-pages/>)

Modelos de Inteligencia Artificial y Soporte Técnico

Modelos de Lenguaje Avanzados (LLMs): Utilizados como asistentes interactivos de arquitectura de software para el filtrado rápido de APIs multimedia complejas, validación conceptual de sincronización de audio en el frontend y optimización de flujos de trabajo DevOps en entornos de despliegue continuo.

(Gemini, Claude, Chatgpt).

20. Agradecimientos

"Para finalizar este proyecto, quiero agradecer personalmente como co-desarrollador a mi compañero de equipo, Eduardo Piquer Sánchez, todo el esfuerzo, las ganas y la motivación que ha demostrado para resolver las tareas más complejas del sistema y superar juntos los días de mayor estrés. Gracias por darme fuerza y apoyo en los momentos difíciles. Sé con total certeza que será un excelente programador, pero de lo que estoy aún más seguro es de la gran persona que es. Muchas gracias por este camino compartido, Eduardo."

— Adrián Vicente López